CAST: 리전 용량 제약을 고려한 탄소 인지 시공간 워크로드 스케줄링

CAST: Carbon-Aware Spatio-Temporal Scheduling under Capacity Constraints

초록 탄소 인지 스케줄링은 작업을 탄소집약도가 낮은 리전과 시각으로 옮기지만, 리전 용량을 두지 않으면 작업이 가장 깨끗한 한 리전에 몰린다. 본 논문은 미래 자리를 확정하지 않고 매시간 대기 작업 전부의 리전과 시작 시각을 선형 계획으로 다시 배정하는 스케줄러 CAST를 제안한다. 8개 리전 1년 시뮬레이션에서 CAST는 마감과 용량 상한을 넘긴 배정 없이, 평균 배출계수로 센 배출을 합성 워크로드에서 71.9%, Azure VM 트레이스에서 51.3% 줄였다. 합성 워크로드에서 다른 작업의 확정 계획을 보지 않고 시각을 정하는 설계는 시작 때 확인하지 않으면 작업의 9.0~17.0%가 상한을 넘겼고, CAST는 도착 때 한 번 정하고 다시 풀지 않는 설계보다 같은 왕복 지연의 고정 가중치에서 1.2%p 덜 배출했다. 선행 정책을 상한 없이 돌리면, CAST의 기본 설정보다 많이 줄이는 둘은 가장 붐빈 리전에 홈 리전 즉시 실행 때 최대 동시 실행의 2.2~9.9배를 요구했다.

주제어: 탄소 인지 스케줄링, 워크로드 이동, 리전 용량, 선형 계획법, 롤링 지평

Abstract Carbon-aware schedulers move jobs to regions and hours with lower carbon intensity, but without regional capacity limits the jobs pile into the single cleanest region. We propose CAST, a carbon-aware scheduler that never commits future slots: every hour it reassigns the region and start time of all waiting jobs with one linear program. In a one-year simulation over eight regions with 2025 carbon-intensity data, CAST missed no deadline, made no assignment above the regional capacity cap, and cut emissions, counted with average emission factors, by 71.9% on a synthetic workload and 51.3% on an Azure VM trace. On the synthetic workload, a design that fixes start times without seeing other jobs' committed plans started 9.0–17.0% of jobs above the cap unless it re-checked capacity at start time, and CAST emitted 1.2 percentage points less than a design that places each job once on arrival and never re-solves it, at a fixed weight matched in round-trip latency. Run without the cap, the two prior policies that save more than CAST's default setting need 2.2 and 9.9 times the home-region peak concurrency in their busiest region.

Keywords: Carbon-aware scheduling, Workload shifting, Regional capacity, Linear programming, Rolling horizon

1. 서론

데이터센터가 쓰는 전력이 빠르게 늘고 있다. 국제에너지기구는 전 세계 데이터센터의 전력 소비가 2024년 약 415 TWh에서 2030년 945 TWh로 늘 것으로 내다본다[1]. 그런데 같은 1 kWh라도 어디서, 언제 쓰느냐에 따라 나오는 탄소가 다르다. 본 연구의 여덟 리전에서 2025년 탄소집약도의 연평균은 인도가 543.2 gCO_2/kWh, 프랑스가 13.6 gCO_2/kWh로 약 40배 차이가 난다. 한 리전 안에서도 캘리포니아는 하루 중 가장 높은 시각이 가장 낮은 시각의 평균 3.93배다. 배치 작업은 대개 마감까지 여유가 있고 클라우드는 이미 여러 리전에 걸쳐 있으므로, 작업을 깨끗한 리전과 시각으로 옮기면 같은 계산을 더 적은 배출로 마칠 수 있다.

그런데 리전 용량을 두지 않으면 옮기는 곳이 한 곳으로 모인다. Sukprasert 등의 상한 분석[2]이 하는 배치를 본 연구의 1년치 자료에 그대로 재현하면 작업 143,200건 가운데 99.75%가 프랑스 한 리전으로 가고, 그 리전은 동시에 129건을 돌려야 한다. 홈 리전에서 즉시 실행할 때의 리전별 최대 동시 실행, 곧 기준 피크는 어느 리전도 13건을 넘지 않는다. 용량을 제약으로 두면 질문은 어느 리전이 깨끗한가가 아니라, 그 리전이 받아 줄 수 있는 만큼만 보내고 남은 작업을 어디에 언제 둘 것인가가 된다. 이 질문이 어려운 것은 가장 깨끗한 리전의 가장 깨끗한 시각이 모든 작업에게 같기 때문이다.

용량을 두는 연구는 각기 다른 것을 양보한다(2장, 표 1). Caspian[3]은 매시간 다시 풀지만 용량을 지키려고 마감을 벌점으로 양보한다. Zanotto 등[4]은 리전 상한과 하드 마감을 함께 두지만 작업이 도착할 때 한 번 자리를 정하고 바꾸지 않는다. Carbon Aware SDK[5] 같은 도구로 작업마다 시작 시각을 따로 고르면 작업들은 서로의 계획을 모른다.

본 논문은 미래 자리를 미리 확정하지 않고 매시간 대기 작업 전부의 리전과 시작 시각을 다시 배정하는 탄소 인지 스케줄러 CAST를 제안한다. CAST는 작업 마감을 지키면서 실험한 모든 조건에서 리전 용량 상한을 넘긴 배정 없이 배출을 줄였다. 본 논문의 기여는 셋이다.

(1) CAST: 매시간 대기 작업 전부의 리전과 시각을 다시 풀어 지금 시작할 작업만 확정한다. 마감과 용량 상한을 지키며 상한 없는 실행 절감의 98%를 얻었다(합성, 4.3절).

표 1 선행 연구의 설계 성질. ○ 있음, △ 벌점·부분 분석·집계 부하, × 없음, — 해당 없음. CAST의 용량 ○는 가장 큰 벌점을 둔 소프트 제약이며 실험에서 초과는 0이었다. COAST의 마감은 권고형이라 실행을 보장하지 않는다.

Table 1 Design properties of prior work. ○ yes, △ penalty, partial analysis or aggregate load, × no, — not applicable. CAST's capacity mark is a soft constraint with the largest penalty; no excess occurred in the experiments. COAST's deadlines are advisory and do not guarantee execution.

Work	Time	Space	Region cap	Hard deadline	Re-solve
Temporal-only [6–8]	○	×	×	△	×
GREEN [9]	○	×	△	×	○
CICS [13]	○	×	△	×	×
CASPER [10]	×	○	△	—	—
Sukprasert [2]	○	○	△	○	×
CarbonClipper [11]	○	○	×	○	—
Hall [12]	○	○	△	△	×
CarbonFlex [14]	○	×	○	△	○
COAST [15]	○	×	△	○	×
Caspian [3]	○	○	○	△	○
Zanotto [4]	○	○	○	○	×
CAST	○	○	○	○	○

(2) 위반을 가르는 것: 다른 작업의 계획을 보지 않고 시각을 정하면 시작 때 확인 없이는 합성 작업의 9.0~17.0%(시드 포함)가 상한을 넘겼다. 계획을 보거나, 다시 풀거나, 시작 때 확인해 빈 리전으로 보내면 0이다(4.2절).

(3) 필요 용량으로 다시 잰 선행 정책: CAST 기본 설정보다 많이 줄이는 두 정책은 상한 없이 돌리면 가장 붐빈 리전에 기준 피크의 2.2배와 9.9배를 요구했다(4.5절).

위반 없는 설계 가운데 가장 가까운, 도착 때 한 번 정하고 다시 풀지 않는 설계보다 CAST는 같은 왕복 지연의 고정 α에서 1.19%p 덜 배출했고(합성), 같은 네 주의 오프라인 하한에도 1.16~2.22%p로 그 설계(2.56~3.63%p)보다 가까웠다(LSTM, 고정 α, 4.4절). 기법 하나하나는 알려져 있다. 본 논문은 이를 마감과 용량을 함께 지키는 매시간 재배정으로 묶고, 용량 확인 방식에 따른 차이를 1년 규모로 잰다. 2장은 관련 연구, 3장은 CAST, 4장은 실험, 5장은 결론이다.

2. 관련 연구

시간 축 연구는 한 데이터센터 안에서 작업을 저탄소 시각으로 미루고[6–9], 공간 축 연구는 작업을 깨끗한 리전으로 보낸다[10]. 두 축을 함께 본 Sukprasert 등은 공간 이동이 지배적이라고 했지만 결합 분석에서는 리전 용량을 두지 않았다[2]. CarbonClipper는 작업 하나를 마감 안에서 리전 사이로 옮기는 온라인 문제에 경쟁비 보장을 주었고[11], Hall 등은 집계 부하를 하루 전 계획과 실시간 배치로 옮긴다[12].

용량을 두는 연구는 셋으로 나뉜다. CICS는 하루 전 자원 곡선 하나로 유연 부하를 묶어 개별 작업의 마감을 보지 못하고[13], CarbonFlex와 COAST는 단일 시설 안의 시간 이동에서 용량을 보지 않으면 작업이 저탄소 시각에 몰린다는 점을 보였다[14, 15]. Caspian은 매시간 다시 풀되 마감을 벌점으로 둔다[3]. Zanotto 등은 리전별 상한과 하드 마감을 두고 도착할 때 기존 배정을 모두 보고 한 번 정하며, 평가는 1개월 동안 1,000건 묶음을 되풀이한 것이다[4]. Caspian의 평가는 작업 200건이다. CASPER[10], CICS[13], Caspian[3]은 모두 목적 사이의 가중치를 사람이 정한다. 표 1에 이 성질을 모았다. 표 1의 열에는 없지만 CAST는 이론 보장과 실제 클러스터 구현을 갖추지 못했다.

3. CAST

CAST는 예측기와 배정 단계로 이루어진다. 예측기는 리전별 향후 24시간의 탄소집약도를 내고, 배정 단계는 매 1시간 슬롯마다 아직 시작하지 않은 작업 전부의 실행 리전과 시작 슬롯을 하나의 선형 계획으로 함께 정한다. 지금 시작하는 작업만 확정해 리전별 실행 기록에 올리고 나머지는 다음 슬롯에 새 예측으로 다시 푼다. 배치는 예측값으로 정하되 배출은 실측값으로 세어 예측 오차의 대가가 결과에 남게 했다(그림 1).

3.1 문제 정의

작업 j는 출발 리전 o_j, 제출 시각 s_j, 실행 시간 d_j, 지연 등급 k_j, 허용 시간(allowance) L_j를 가지며 마감은 $D_j = s_j + L_j + d_j$다. L_j는 시작을 미룰 수 있는 시간이자 작업별 왕복 지연(RTT) $\ell(o, r)$의 상한을 겸해, $\ell(o_j, r) > L_j$인 리전은 후보에서 뺀다. 리전 r은 동시 실행 수의 용량 상한 cap_r을 가진다. 정하는 값은 실행 리전 $r(j)$와 시작 시각 τ_j이며, $n_r(t)$는 리전 r에서 시각 t에 도는 작업 수다. 목적은 사전식이다. 상한을 넘긴 양 Z를 먼저 최소화하고, 그

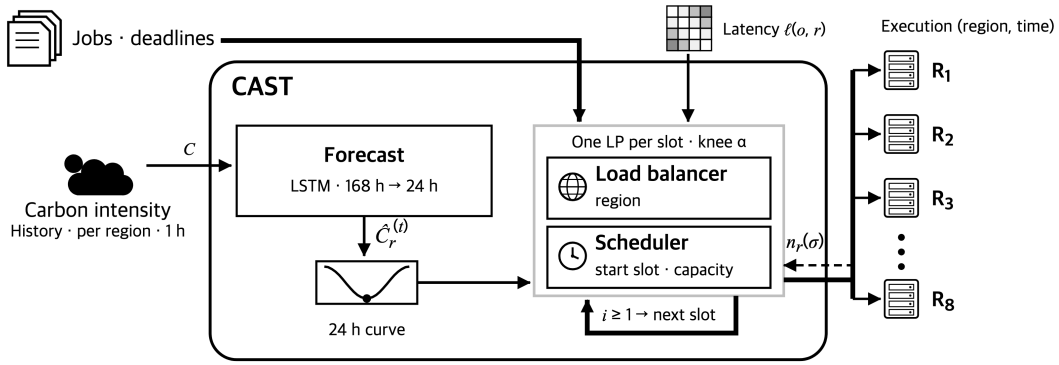


그림 1 CAST의 파이프라인. 굵은 선은 작업, 가는 선은 탄소·왕복 지연 자료, 파선은 리전별 점유가 배정 단계로 돌아가는 길이며, 회색 테두리 안의 로드밸런서(리전 선택)와 스케줄러(시각 선택)는 매 슬롯 하나의 선형 계획으로 함께 풀린다.

Fig. 1 Pipeline of CAST. Thick lines carry jobs, thin lines carbon and round-trip latency data, and the dashed line returns each region's occupancy to assignment; the load balancer (region) and scheduler (time) inside the gray frame are solved as one linear program per slot.

위에서 왕복 지연 Λ가 왕복 지연 한도 Λ를 넘지 않는 배정 가운데 배출 E를 최소화한다.

$$E = \sum_{j\in J} P \int_{\tau_j}^{\tau_j+d_j} C_{r(j)}(t)dt \qquad (1)$$

$$Z = \sum_{r\in R} \int \max\{0, n_r(t) - \mathrm{cap}_r\}dt \qquad (2)$$

$$s_j \le \tau_j, \quad \tau_j + d_j \le D_j \qquad \forall j \in J \qquad (3)$$

$$n_r(t) \le \mathrm{cap}_r + z_r(t),\ z_r(t) \ge 0 \quad \forall r \in R,\ \forall t \qquad (4)$$

$C_r(t)$는 실측 탄소집약도, P는 작업 전력이다. Λ는 각 작업의 왕복 지연에 실행 시간을 곱해 더한 값으로, 원격 리전을 점유하는 동안 자료가 오가고 장애에 노출되는 비용을 뜻한다. 마감은 하드 제약(식 (3))이고, 용량은 초과 $z_r(t)$를 변수로 두어 그 적분 Z를 가장 먼저 최소화하므로(식 (2), (4)) 늘 해가 있다. 1년을 한 번에 풀려면 모든 작업과 탄소집약도를 미리 알아야 하므로, 이 문제는 참조로 두고 매 슬롯 3.3절의 문제를 푼다. 제출 슬롯 말고는 시작할 정수 슬롯이 없는 작업($\lfloor D_j - d_j \rfloor \le \lfloor s_j \rfloor$)을 미룰 수 없는 작업이라 부른다.

3.2 예측기

예측기는 리전마다 독립인 2층 LSTM(은닉 64, 드롭아웃 0.2)이다. 과거 168시간의 탄소집약도, 무탄소·재생 전력 비율, 시각·요일·월의 주기 변수, 공휴일 여부를 받아 향후 24시간을 한 번에 낸다. 캘리포니아, 텍사스, 독일은 풍속, 일사량, 기온(Open-Meteo 과거 기상 API의 시간 단위 값)을 더하고 향후 24시간의 같은 변수도 넣는데, 이는 예보가 아닌 미래 기상 실측값이다.

학습은 2021~2023년, 검증은 2024년, 시험은 2025년으로 연도를 끊었고, 스케줄링 실험은 시험 연도 위에서만 돈다. 입력은 학습 연도에만 맞춘 최솟값–최댓값 척도로 0~1에 옮긴다. 손실은 정규화한 값의 평균 제곱 오차이고, Adam(학습률 0.001, 가중치 감쇠 0.0001)으로 배치 64씩 학습하며 기울기 노름을 1.0으로 자른다. 검증 오차가 8 학습 회차(epoch) 동안 줄지 않으면 학습률을 절반으로 낮추고, 15 회차 동안 줄지 않거나 가장 좋았던 회차 뒤 30 회차가 지나면 멈추되, 최소 20 회차, 최대 150 회차를 돈다. 모델은 원래 단위로 되돌린 검증 평균 절대 오차가 가장 작은 회차의 것을 쓰며, 난수 시드는 42다.

2025년 평균 절대 오차는 26.10 gCO_2/kWh로 24시간 전 값을 그대로 쓰는 예측(SN-24h, 25.27)과 비슷하며, 이 차는 하루 변동이 2024년 11월부터 두 배가 된 인도 하나에서 오고 인도를 빼면 LSTM 25.47 대 27.24다. 세 리전의 예측을 24시간 전 값으로 바꾸어도 절감은 0.37%p 움직인다(표 5). 하루 앞 탄소집약도 예측은 따로 연구된 문제이며[16] 본 연구의 기여가 아니다.

3.3 매 슬롯 배정

매 슬롯 t마다 아직 시작하지 않은 작업을 모아 대기 집합 Q를 만든다. 슬롯 t에 제출되는 작업은 슬롯 시작에 모두 안다고 보고 함께 배정한다(최대 1시간의 선견이며 모든 비교 설계에 같다). 후보는 리전 r과 시작 오프셋 i의 쌍이다. i=0은 지금, 곧 $\max(t, s_j)$에 시작하는 것이고, i의 상한은 마감과 예측 지평 H=24 가운데 이른 쪽인 $h_j = \lfloor \min(D_j - d_j, t + H) - t \rfloor$다. 미룰 수 없는 작업과 여유가 한

슬롯보다 적은 작업은 i=0만 허용한다. 후보의 비용은 정규화한 탄소와 왕복 지연의 가중합에 실행 시간을 곱한 값이다.

$$c_j(r,i) = d_j\left[\alpha \tilde{C}_j(r,i) + (1-\alpha)\tilde{\ell}(o_j,r)\right] \quad (5)$$

$$\tilde{C}_j(r,i) = \frac{\hat{C}_r^{(t)}(t+i, d_j)}{\max_{r',t'} \hat{C}_{r'}^{(t)}(t')}, \qquad \tilde{\ell}(o,r) = \frac{\ell(o,r)}{244} \quad (6)$$

$\hat{C}_r^{(t)}(t+i,\ d_j)$는 슬롯 t에 발행된 예측에서 $t+i$부터 d_j 동안의 평균 탄소집약도이며(지금 슬롯의 첫 한 시간은 실측), 분모는 그 슬롯 예측의 최댓값이고 왕복 지연은 행렬의 최댓값 244 ms로 나눈다. 탄소 항을 슬롯마다 정규화하므로 같은 α라도 슬롯마다 kg과 ms의 교환비가 다르다. 슬롯 t의 배정 문제는 다음과 같다.

$$\min \sum_{j\in Q}\sum_{r,i} c_j(r,i)x_{jri} + M_A\sum_{r,\kappa} z^A_{r\kappa} + M_B\sum_{r,\sigma\geq t+1} z^B_{r\sigma} + \varepsilon\sum_{j\in Q}\sum_{r,i} i\, x_{jri} \quad (7)$$

$$\sum_{r,i} x_{jri} = 1, \qquad \forall j \in Q \quad (8)$$

$$\sum_{(j,0)\ni u} x_{jr0} + n_r(u) \leq \mathrm{cap}_r + z^A_{ru} \quad \forall r,\ u \in U_t \quad (9a)$$

$$\sum_{(j,i)\ni \sigma} x_{jri} + n_r(\sigma) \leq \mathrm{cap}_r + z^A_{r\sigma} \quad \forall r,\ \sigma \in [t+1, t+H] \quad (9b)$$

$$\sum_{(j,i)\ni\sigma,\ j\in Q^{\mathrm{def}}} x_{jri} + n_r(\sigma) \leq \mathrm{cap}_r - \rho_r + z^B_{r\sigma} \quad \forall r,\ \sigma \in [t+1, t+H] \quad (10)$$

$0 \leq x_{jri} \leq 1$은 작업 j를 리전 r에서 i슬롯 뒤에 시작하게 두는 변수이고, 식 (8)은 한 작업을 한 자리에 둔다. 지금 슬롯의 용량 제약(9a)은 작업이 시작하는 순간 $u \in U_t$마다 한 행을 두어 그 순간 도는 작업을 실제 시각으로 센다. 미래 슬롯의 용량 제약(9b)은 슬롯 σ를 조금이라도 덮는 실행을 한 자리로 세는 보수적 근사다. 지금 슬롯만 순간 단위로 세는 것은 1초짜리 작업 수십 건이 한 시간 자리를 통째로 차지한 것처럼 세지 않기 위해서다. $n_r(\cdot)$는 이미 시작한 작업의 수로 실행 기록에서 읽는다. 식 (10)은 미룰 수 있는 작업이 미래 슬롯을 끝까지 채우지 못하게 ρ_r 자리를 남기는 예약으로, 선택 사항이다(4.1절).

초과에는 $M_A \gg M_B \gg 1$의 벌점을 두어 물리 초과, 예약 침범, 탄소와 왕복 지연, 이른 시작(ε=10^{-4}) 순으로 본다. M_B를 가장 긴 실행 시간의 2배, M_A를 그 100배로 둔 것은 보장이 아니라 실용적 선택이다. 곧 CAST는 용량을 강제하지 않고 초과를 가장 무겁게 벌하며, 생긴 초과는 모두 보고한다. 먼저 x를 연속으로 두고 선형 계획으로 푼다. 슬롯 경계를 넘는 작업이 없다면 제약 행렬이 완전 단모듈이라 해가 곧 정수해이지만[17–19], 실제로는 이 조건이 깨지므로 분수해가 난 슬롯(3.9%)만 정수 계획으로 다시 푼다. 풀고 나면 i=0인 작업만 확정해 실행 기록에 올리고, 나머지는 다음 슬롯에 다시 푼다. 구성상 세 가지가 성립한다. 후보가 마감 이내로만 잡히므로

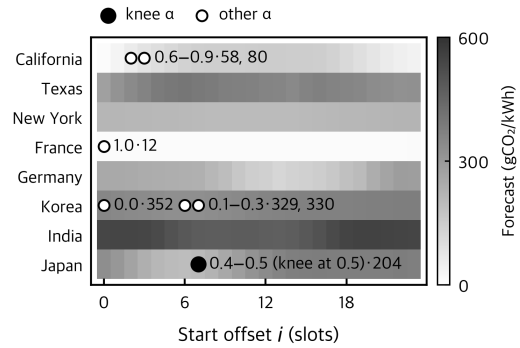


그림 2 한 슬롯에서 α에 따른 배정. 2025년 5월 26일 19시 48분(UTC)에 한국에서 제출된 k=1 작업(실행 17분)이며 칸은 리전 × 향후 24 슬롯, 진할수록 예측 탄소집약도가 높다. ●는 무릎점(α=0.5), ○는 다른 α가 고른 칸이고 옆 숫자는 그 α의 범위와 예측값(gCO₂/kWh)이다.

Fig. 2 Assignment by α at one slot. A k=1 job submitted in Korea at 19:48 UTC on 26 May 2025 (17 min run); cells are region × next 24 slots, darker for higher forecast intensity. The filled dot is the knee (α=0.5) and hollow dots other α, labelled with their α range and forecast intensity (gCO₂/kWh).

마감을 넘긴 배정이 없고, 여유가 슬롯마다 1씩 줄어 어떤 작업도 무한히 미뤄지지 않으며, 초과 변수가 있어 늘 해가 있다.

3.4 가중치 규칙과 한 슬롯의 예

식 (5)의 가중치 α는 탄소와 왕복 지연의 맞바꿈을 정한다. α=1이면 탄소만 보고, α=0이면 홈 리전 즉시 실행이다. CAST는 α를 손으로 정하지 않는다. 슬롯마다 후보 $\mathcal{A}$ = {0, 0.1, …, 1}의 각 α로 식 (7)~(10)을 풀어 실행 시간 가중 평균 왕복 지연 $\bar{\lambda}_\alpha$와 예측 배출 $\bar{e}_\alpha$를 얻고, 두 축을 후보들 사이에서 0~1로 정규화해 원점에 가장 가까운 후보를 고른다.

$$\alpha^* = \operatorname*{argmin}_{\alpha\in\mathcal{A}} \sqrt{\tilde{\lambda}_\alpha^2 + \tilde{e}_\alpha^2} \quad (11)$$

이는 이상점까지의 거리로 고르는 타협해[20]이며 편의상 무릎점[21]이라 부른다. 물리 초과가 생기는 후보가 있으면 초과가 최소인 후보 안에서 고르고, 거리가 같으면 작은 α를, 한 축이 한 점으로 줄면 α=1을 둔다. 이 규칙은 1년치 자료 없이 매시간 그 시각의 예측으로 가중치를 정하며, 그 대가는 4.6절에서 잰다.

그림 2는 한 슬롯에서 한국을 출발한 작업 하나의 배정이 α에 따라 바뀌는 모습이다. α가 0이면 한국에서 지금(352 gCO₂/kWh), 0.1~0.3이면 한국에서 6~7시간 뒤(329~330), 0.4~0.5이면 왕복 지연 30 ms를 들여

표 2 합성 워크로드의 지연 등급별 허용 시간과 실행 시간. 허용 시간은 등급 안에서 로그균등, 실행 시간은 로그정규에 상·하한을 두어 뽑는다(초 단위, 생성 모수). 건수는 1년 전체다.

Table 2 Allowance and duration by tier (synthetic). Allowances are log-uniform within each tier; durations are log-normal and clamped (seconds; generation parameters). Counts cover the full year.

k	Allowance	Median d	s.d. (log)	Clamp	Jobs
1	12–24 h	5,400	0.9	300–21,600	43,830
2	2–8 h	1,800	0.9	60–14,400	14,458
3	2–10 min	180	0.8	5–1,800	14,716
4	15–60 s	30	0.7	1–300	29,244
5	1–5 s	5	0.6	1–60	43,752

일본에서 7시간 뒤(204), 0.6~0.9이면 캘리포니아에서 2~3시간 뒤(58~80), 1이면 프랑스에서 즉시(12) 시작한다. α 하나가 리전과 시작 시각을 함께 움직이며, 무릎점은 이 슬롯에서 0.5를 골랐다. 알고리즘 1이 한 슬롯의 절차다.

알고리즘 1 결합 롤링 배정

Algorithm 1 Joint rolling assignment

Input: cap_r, ρ_r, H, forecasts $\hat{C}_r^{(t)}(\cdot)$, job stream
Output: $(r(j), \tau_j)$ for all j

$Q \leftarrow \emptyset$
for $t \leftarrow 0, 1, 2, \ldots$ do
  for each j with $\lfloor s_j \rfloor = t$ do $Q \leftarrow Q \cup \{j\}$
  for each $j \in Q$ do
    $h_j \leftarrow \lfloor \min(D_j - d_j,\ t + H) - t \rfloor$
    if j is non-deferrable or $(D_j - d_j) - t < 1$ then $h_j \leftarrow 0$
  for each $\alpha \in \{0, 0.1, \ldots, 1\}$ do
    $x^\alpha \leftarrow$ LP solution of (7)–(10)
    $(\bar{\lambda}_\alpha, \bar{e}_\alpha) \leftarrow$ duration-weighted mean RTT and forecast emissions of x^α
  $\alpha^* \leftarrow$ knee point by (11) with the tie rules of Sec. 3.4; $x \leftarrow x^{\alpha^*}$
  if x is fractional then $x \leftarrow$ MILP solution of (7)–(10) at α^*
  for each $j \in Q$ with $x_{jr0} = 1$ do
    $r(j) \leftarrow r$; $\tau_j \leftarrow \max(t, s_j)$; record $[\tau_j,\ \tau_j + d_j)$ in ledger;
    $Q \leftarrow Q \setminus \{j\}$

4. 실험

4.1 설정, 회계, 비교 설계

평가는 8개 리전의 2025년 탄소집약도 위에서 한다. Electricity Maps[22]가 사후 확정한 1시간 해상도의 소비 기준 값[23]이며, 프랑스의 최솟값이 2 gCO_2/kWh여서 직접 배출 기준으로 보인다. 리전과 구역은 한국 KR, 캘리포니아 US-CAL-CISO, 텍사스 US-TEX-ERCO, 뉴욕 US-NY-NYIS, 프랑스 FR, 독일 DE, 인도 IN, 일본 JP이고, 왕복 지연은 같은 순서의 Azure 리전 사이 실측 행렬(12~244 ms)[24]에서 읽는다. 뉴욕은 Azure East US(버지니아)의 지연과 뉴욕 전력망의 탄소를 섞은 것이다. 2025년 1월 1~7일은 LSTM 입력을 채우는 준비 구간이라, 지표는 1월 8일부터 8,592시간에 대해 센다.

워크로드는 둘이다. 합성 워크로드는 리전당 하루 50건, 평가 구간의 작업 143,200건이며 다섯 지연 등급의 허용 시간과 실행 시간은 표 2를 따른다. 미룰 수 없는 작업이 건수로 59.1%지만 작업시간으로는 1.3%다. Azure 워크로드는 2019년 VM 트레이스[25, 26]에서 29일씩 13장을 이어 1년 146,016건을 만들었다. 실행 시간은 VM 수명 그대로이고, 출발 리전은 균등 무작위로 주었으며, VM 크기는 쓰지 않아 VM 시간으로 가중한 값이다. 평균 동시 실행은 총 상한 대비 합성 8.4%, Azure 26.1%다.

용량 상한은 기준 피크(합성 13, Azure 34)의 1.25배를 올림한 17과 43을 모든 리전에 같게 둔다. 기준 피크는 평가 구간에서 잰 값이므로, 상한을 하나로 고정하지 않고 0.5~3배의 결과를 함께 보고한다(4.3절, 표 5). 예약은 합성 3, Azure 0이다. 합성의 3은 준비 구간에서 잰 미룰 수 없는 작업의 최대 동시 실행이며, Azure에서 같은 방법으로 잰 13은 넘어와 아직 도는 긴 VM을 두 번 세는 과대값이라 대표로 쓰지 않는다.

배출은 실행 리전의 실측 강도를 실행 구간에 대해 슬롯 단위로 적분한 값에 작업당 1 kW를 곱해 센다. 겹침 길이로 정확히 적분해도 무릎점 절감은 0.13%p 움직인다. 유휴·냉방 전력과 설비의 내재 배출은 넣지 않는다. 지표는 홈 리전 즉시 실행 대비 절감률, 위반(작업이 시작하는 순간 같은 리전에서 이미 상한만큼 돌고 있으면 그 작업을 셈), 필요 용량(그 배정이 위반 없이 돌려면 필요한 상한, 곧 리전별 최대 동시 실행의 최댓값), 대기 시간(waiting time, 제출에서 시작까지)이다. %p는 반올림 전 kg 차를 기준 배출로 나눈 값이다.

두 설계를 견줄 때는 같은 왕복 지연 짝을 쓴다. 비교 대상의 실행 시간 가중 평균 왕복 지연과 0.5 ms 안에서 같은 값을 내는 고정 α를 이분법으로 찾아 1년을 돌리고, 차의 95% 신뢰구간은 7일 블록 부트스트랩 2,000회로 낸다. 비교 설계는 모두 CAST와 같은 선형 계획, 예측, 비용, 풀이기 위에 두며 표 3에 설계 요소를 적었다. 다시 풀지 않는 설계는 Zanotto 등[4]의 방식, 곧 도착할 때 이미 정한 배정을 모두 보고 리전과 시각을 한 번 정한 뒤 다시 풀지 않는 결정을 CAST의 선형 계획 위에 옮긴 것이다. 빈 리전으로 보내는 설계는 다른 작업의 확정 계획을 보지 않고 시각을 정하되, 시작 때 확인해 자리가 없으면 그 순간 자리가 있는 리전 가운데 예측 탄소가 가장 낮은 곳으로 보낸다. 실험은 SciPy 1.18.0의 HiGHS로 Apple M1 Max(메모리 32 GB)에서 돌렸고, 코드와 실행 설정은 심사용 익명 저장소(투고 때 기재)에 둔다.

표 3 비교 설계의 요소, 위반, 배출 차. 위반은 평가 구간 작업 대비 %, Δ는 그 설계 배출 − CAST 배출(기준 대비 %p, 양수면 CAST가 덜 배출)이며 실행 시간 가중 왕복 지연을 맞춘 고정 α 짝이다. Azure는 예약 ρ=0이며, ρ=13에서는 다시 풀지 않는 설계의 Δ가 −2.10이다. † 같은 α(0.5375), ‡ 예약 없는 CAST와의 차로 지연을 맞추지 않음.

Table 3 Design features, violations and emission gaps. Viol. is % of evaluated jobs; Δ is that design's emissions minus CAST's (pp of baseline; positive = CAST emits less) at a fixed α matched in duration-weighted RTT. Azure uses ρ=0; at ρ=13 the no-re-solve Δ is −2.10. † Same α (0.5375). ‡ Versus CAST without reservation, latency not matched.

Design	Re-solve	Sees others' plans	Start check	Cap constraints	Syn. viol. (%)	Syn. Δ pp (95% CI)	Azure viol. (%)	Azure Δ pp (95% CI)
CAST	○	○	—	now+future	0	—	0	—
No re-solve [4]	×	○	—	now+future	0	+1.19 (0.63–1.81)	0	+2.53 (1.68–3.42)
Plan-blind	×	×	×	started only	9.0–17.0	n/a	11.4–14.7	n/a
Plan-blind + check (origin)	×	×	origin	started only	0.16–0.56	n/a	5.2–5.3	n/a
Plan-blind + check (free)	×	×	free	started only	0	+0.68† (0.46–0.91)	0	+1.99 (0.86–3.01)
Now-only CAST	○	○	—	now	0	+0.67‡	0	+0.12‡
No cap	○	—	—	none	17.5	—	42.9	—

4.2 용량 확인 방식과 위반

위반은 용량을 언제, 무엇과 함께 확인하느냐로 갈린다(표 3). CAST, 다시 풀지 않는 설계, 미래 슬롯 용량을 보지 않는 CAST, 빈 리전으로 보내는 설계는 두 워크로드와 모든 예측기에서 위반이 0이다. 다른 작업의 확정 계획을 보지 않고 시각을 정하면서 시작 때 확인하지 않는 설계는 평가 구간 작업의 9.0~17.0%(시드 포함, Azure 11.4~14.7%)가 상한을 넘긴 순간에 시작한다. 시작 때 확인하되 여유가 다한 작업을 원래 리전에 넣으면 227~387건(시드 포함 0.16~0.56%, Azure 5.2~5.3%)이 남는다. 이 위반은 모두 꽉 찬 원 리전에 넣는 규칙에서 났고, 그 순간 다른 리전에는 늘 빈자리가 있었다.

곧 위반 0에 이르는 길은 셋이다. 다른 작업의 계획을 모두 보거나, 확정하지 않고 다시 풀거나, 시작 때 확인해 빈 리전으로 보내는 것이다. CAST의 물리 초과 변수 z^A는 두 워크로드 모두 양수가 된 적이 없고, 예약 침범 z^B는 Azure(ρ=13)에서 슬롯의 52.6%에서 양수였다. 슬롯 안 도착을 미리 알지 않고 도착 순간에 배정해도 CAST의 위반은 0이고 절감은 0.56~1.62%p 준다.

4.3 용량을 지키는 비용

합성 워크로드에서 CAST의 무료점은 71.93%를 줄여 용량 상한 없음(73.39%)의 98%를 얻는다. α=1에서는 95.65% 대 95.97%다. 상한 없음은 그 대가로 위반이 24,993건과 46,354건이고, 최대 동시 실행이 63건과 117건으로 상한의 3.7배와 6.9배다. CAST는 평가 구간 작업의 58.4%를 출발 리전 밖에서 돌렸고, 실행 묶은 프랑스 51.6%, 캘리포니아 27.3%다. 상한을 기준 피크의 0.5, 1, 2, 3배(7, 13, 26, 39)로 바꾸면 무료점 절감은 56.79%, 70.16%, 73.21%, 73.42%이고 모두 위반이 0이다. 상한 7에서는 홈 리전 즉시 실행조차 713번 상한을 넘긴다. 용량을 지키는 비용은 상한 13에서 3.24%p, 17에서 1.47%p, 26에서 0.19%p로 2배 근처에서 거의 사라지고, 도착을 두 배로 늘리면 3.85%p다.

Azure(ρ=0)에서는 비용이 훨씬 크다. CAST 무료점 51.35%에 상한 없음은 62.70%이며(두 실행의 왕복 지연이 13 ms 다르다), α=1에서는 67.74% 대 95.11%다. 평균 동시 실행 89.9건이 가장 깨끗한 프랑스의 상한 43을 넘기 때문이다. 상한을 기준 피크의 2배(68)로 늘리면 비용이 11.35%p에서 0.80%p로 준다(ρ=0).

4.4 가장 가까운 설계와의 비교

표 3의 Δ는 같은 왕복 지연 짝의 배출 차다. 합성 워크로드에서 다시 풀지 않는 설계는 CAST보다 1.19%p(95% 신뢰구간 0.63~1.81) 더 배출하고, 빈 리전으로 보내는 설계는 같은 α(0.5375)에서 0.68%p(0.46~0.91, 실행 시간 가중 왕복 지연 차 0.35 ms) 더 배출한다. 미래 슬롯의 용량 제약은 위반을 막는 것이 아니라 붐빌 슬롯을 미리 피해 0.67%p를 더한다(예약 없는 CAST와, 지연을 맞추지 않음). 선형 계획 대신 작업마다 차례로 자리를 잡는 탐욕으로 풀면 0.84%p 더 배출한다. 운용 규칙인 무료점의 CAST(71.93%)는 같은 왕복 지연의 고정 α로 돈 다시 풀지 않는 설계(α=0.5862, 72.08%)와 배출이 같고(신뢰구간이 0을 포함) 대기 시간은 1.47 대 2.13시간이다.

Azure(ρ=0)에서는 CAST(51.35%)가 같은 왕복 지연 짝으로 다시 풀지 않는 설계보다 2.53%p(1.68~3.42) 덜 배출하지만 대기 시간은 CAST가 더 길다(2.51 대 1.80시간). 빈 리전으로 보내는 설계보다는 1.99%p(0.86~3.01) 덜 배출한다. 예약 ρ=13에서는 다시 풀지 않는 설계가 2.10~3.62%p 앞선다(시드를 다시 뽑으면 최대 5.71). 예약 행이 CAST의 절감만 5.20%p 깎기

표 4 선행 정책과 필요 용량(합성, 회계 동일). 위반은 상한 17 기준, 필요 용량은 리전별 최대 동시 실행의 최댓값(괄호는 시간 가중 99.9 백분위수), Total은 리전별 최댓값의 합이다. 바꾼 점: a 작업 수로 셈, 시간별 허용량 ≤ 하루 평균의 2배, 첨두 가중치 0. *b* 고정 여유 대신 작업별 마감. *c* 원문 코드의 가중치 0.9, 서버당 10건, 리전당 100대. *d* 공개 코드 없음, 슬롯 안 자원량 = 실행 시간, 마감을 1시간 넘긴 작업은 홈에서 즉시 시작.

Table 4 Prior policies and required capacity (synthetic, same accounting). Viol. is against cap 17; Req. cap is the largest per-region peak concurrency (time-weighted 99.9th percentile in parentheses); Total sums per-region peaks. Changes: a job counts, hourly allowance ≤ 2× daily mean, peak weight 0; b per-job deadlines instead of fixed slack; c original code's weight 0.9, 10 jobs per server, 100 servers per region; d no public code, per-slot resource = run time, jobs 1 h past deadline start at home.

Policy	Knows ahead	Savings (%)	Viol. @17	Req. cap (p99.9)	Total	Deadl. miss	RTT (ms)
Baseline	—	0	0	13 (8)	89	0	0
CICS^a [13]	actual, next day	13.89	0	12 (9)	88	0	0
Suk. temporal, home^b [2]	actual, 1 yr	22.22	3,180	27 (19)	194	0	0
Suk. temporal, CAST regions^b	actual, 1 yr	73.23	37,236	82 (64)	267	0	47.8
CASPER^c [10]	actual, this hour	95.30	11,061	29 (24)	46	0	120.4
Suk. spatial+temporal^b	actual, 1 yr	96.63	61,539	129 (107)	139	0	120.4
Caspian^d [3]	LSTM, 24 h	67.54	1,538	34 (28)	86	5,582	115.7
CAST, knee	LSTM, 24 h	71.93	0	17 (17)	117	0	47.8
CAST, α=1	LSTM, 24 h	95.65	0	17 (17)	46	0	120.0

때문이며, 새로 도착하는 몫에 맞춘 ρ=6에서는 CAST 50.44% 대 다시 풀지 않는 설계(ρ=0) 49.04%다.

매시간 다시 푸는 온라인 구조가 얼마를 내주는지는 계절이 다른 네 주를 한 문제로 푼 오프라인 하한과 견준다. 하한은 같은 왕복 지연 한도에서 완전예지 실측으로 그 주 작업을 한 번에 배정하며, 슬롯을 통째로 덮는 실행만 세는 완화라 실제 최적보다 낮을 수 있다. 하한과의 격차는 CAST가 0.13~0.64%p여서(완전예지, 고정 α) 이 부하에서 CAST의 온라인 구조는 얻을 수 있는 최선에 가깝다. LSTM으로 재면 CAST(α=0.5375)는 1.16~2.22%p, 다시 풀지 않는 설계(α=0.5437)는 2.56~3.63%p로 네 주 모두 CAST가 하한에 더 가깝다(고정 α). 무릎점의 격차는 3.11~6.49%p(LSTM)이며, 그 차는 예측 오차와 무릎점 규칙에서 온다.

4.5 선행 정책과의 비교

선행 정책 넷을 원문 수식대로 본 연구의 자료 위에 다시 구현하고(바꾼 점은 표 4 캡션) 같은 회계로 채점했다. 상한 17은 본 연구가 고른 값이므로, 선행 정책은 상한 없이 돌리고 그 배정이 위반 없이 돌려면 필요한 상한, 곧 필요 용량으로 견준다(표 4). CAST 무릎점보다 많이 줄이는 둘은 CASPER(95.30%)와 Sukprasert 공간+시간(96.63%)이며, 가장 붐빈 리전에 29건과 129건, 곧 기준 피크의 2.2배와 9.9배를 요구한다. Sukprasert는 운영 정책이 아니라 1년치 탄소집약도를 미리 아는 상한 분석이다. 탄소만 보는 CAST(α=1)는 상한 17 안에서 95.65%로 CASPER와 비슷하다(회계에 따라 부호가 바뀌는 작은 차).

Sukprasert의 시간 이동을 CAST가 정한 리전 위에서 돌리면 73.23%로 홈 리전 위(22.22%)보다 51.01%p 높다. 이 차는 공간 이동의 몫이며 필요 용량은 82(6.3배)로 다시 몰린다. 이 판은 본 연구가 섞은 것이라 선행 정책 수에 넣지 않는다. Caspian은 마감을 벌점으로 두어 작업의 3.90%가 마감을 넘긴다. 1시간 미만 작업을 실행 시간만큼의 분수로 세므로 자기 단위로는 상한을 지키지만, 시작 순간 단위로 다시 세면 1,538건이 넘친다. 탄소 가중치를 0.5~2로 바꿔도 절감은 67.34~67.97%다. 리전별 최댓값을 모두 더한 설비 총량으로는 CASPER(46)가 CAST 무릎점(117)보다 작다. CASPER는 프랑스 한 곳에 29를 요구하는 대신 나머지를 비운다. 리전마다 용량이 정해져 있다는 본 연구의 가정에서는 한 리전의 크기가 맞는 자다.

4.6 가중치 규칙의 대가

그림 3은 CAST의 고정 α 열한 점과 무릎점이다(합성, LSTM). 무릎점(47.48 ms, 71.93%)은 같은 실행 시간 가중 왕복 지연의 고정 α=0.575(47.08 ms, 73.79%)보다 1.87%p 더 배출한다. 이런 짝을 합성 아홉 조건에서 찾으면 무릎점은 여덟에서 더 배출한다. 24시간 전 값 1.88, 완전예지 2.20, 상한 10 2.64, 시드 넷 1.77~2.28%p이고, 상한 7에서만 0.82%p로 차이가 없다. Azure에서는 차이가 없다(0.63%p, 신뢰구간이 0을 포함, ρ=0). 슬롯이 고른 α는 0.6과 0.5가 89%이며, 예측 곡선의 골이 깊을수록 탄소 축의 정규화가 달라져 무릎점이 지연 쪽으로 물러선다.

고정 α를 고르려고 1년을 돌려야 하는 것은 아니다. 평가 구간의 첫 주만 보고 고른 고정 α=0.5625도 연간 71.92%로 무릎점과 같은 절감을 낸다. 그래서 운영

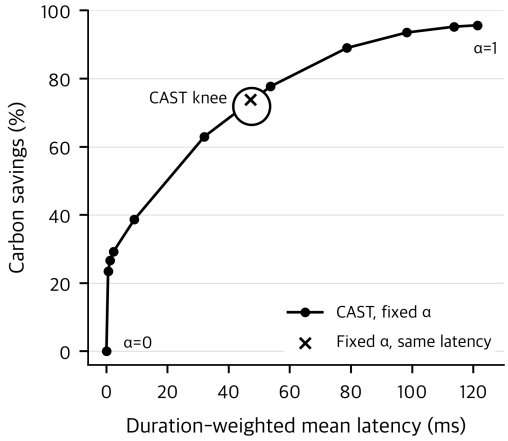


그림 3 CAST의 고정 α와 무릎점(합성, LSTM). ●와 선은 고정 α 열한 점(0, 0.1, …, 1), 과녁은 무릎점(47.48 ms, 71.93%), ×는 무릎점과 같은 실행 시간 가중 왕복 지연을 내는 고정 α(0.575: 47.08 ms, 73.79%)이며 과녁 안에 있다.

Fig. 3 Fixed α and the knee for CAST (synthetic, LSTM). Dots and line are the eleven fixed α (0, 0.1, …, 1), the target is the knee (47.48 ms, 71.93%), and the cross, inside the target, is the fixed α with the same duration-weighted RTT (0.575: 47.08 ms, 73.79%).

지침은 이렇다. 첫 주 자료가 있으면 고정 α가 낫고, 없으면 무릎점이 1년치 자료 없이 매시간 그 시각의 예측으로 가중치를 정하며 그 대가는 약 2%p다.

4.7 결론이 유지되는 범위와 계산 비용

표 5는 조건을 바꾼 결과다. 모든 행에서 위반은 0이다. 절감이 크게 줄어드는 조건은 둘이다. 왕복 지연 한도를 100 ms로 두면 한국, 일본, 인도의 작업이 깨끗한 리전에 닿지 못해 무릎점 절감이 32.98%로 꺾인다. 한계 배출로 세면 절감이 절반 가까이 준다. 프랑스로 옮긴 작업만 같은 시각 독일의 평균 강도로 센 한쪽 대리에서는 다시 풀기의 이점도 사라지지만(37.53% 대 37.55%), 모든 강도를 화석 몫으로 센 대칭 대리에서는 남는다(같은 왕복 지연의 고정 α 짝으로 CAST 34.84% 대 다시 풀지 않는 설계 33.78%). 옮긴 작업마다 출발지에서 전송 에너지를 세면 작업당 약 9 GB, 곧 계산 에너지 1 kWh당 12.9 GB에서 CAST 무릎점이 홈 리전에서 시각만 옮긴 실행(18.78%)과 같아진다.

CAST는 매 슬롯 선형 계획을 열한 번 푼다. HiGHS로 슬롯당 평균 223 ms, 95 백분위수 380 ms, 최대 1.2초가 걸렸고(정수 계획 상대 허용오차 0.01%), 최대 변수는 7,848개, 1년 전체는 34분이다. 문제 크기는 작업 수가 아니라 대기열 길이에 매여, 도착을 두 배로 늘리면 최대 변수가 두 배, 풀이 시간이 2.5배가 되고 Azure는 52분(ρ=13)이다.

표 5 결론이 유지되는 범위(합성, CAST). 모든 행에서 위반은 0이다. Cost of cap은 상한 없음 무릎점 − CAST 무릎점(%p)으로 음수면 CAST가 덜 배출하며, —는 돌리지 않은 칸이다.

Table 5 Robustness (synthetic, CAST). No row has a violation. Cost of cap is the no-cap knee saving minus CAST's knee saving (pp), negative when CAST emits less; — marks runs not performed.

Condition	Knee (%)	α=1 (%)	Cost of cap (pp)
Main (LSTM, cap 17)	71.93	95.65	1.47
Forecaster: SN-24h	64.44	95.59	1.86
Forecaster: oracle	67.20	95.87	2.05
5 workload draws	71.51±0.43	95.40±0.28	1.83±0.53
Cap 7 (0.5×)	56.79	76.30	16.61
Cap 13 (1×)	70.16	94.18	3.24
Cap 26 (2×)	73.21	95.90	0.19
Cap 39 (3×)	73.42	95.96	−0.03
Per-region caps (3)	63.75–73.05	—	0.35–9.64
Arrivals ×2 (cap 23)	69.45	92.58	3.85
Unequal arrival rates	74.53	95.42	0.71
Deferrable share 20–60%	70.01–72.54	—	1.10–3.09
Arrival peak at noon	70.26	—	0.31
Horizon 6–12 h	71.04–71.63	—	0.93–1.18
No carbon lookahead	69.50	95.48	2.54
SN-24h in 3 weather regions	71.55	95.66	1.32
RTT cap 100 ms	32.98	46.83	0.11
Transfer 1 GB, dest. / origin	71.16 / 66.01	95.20 / 87.65	1.46
Life-cycle approx.	67.07	90.28	1.33
Marginal: France one-sided	38.99	18.11	1.03
Marginal: symmetric fossil	36.15	61.51	0.04

5. 결론 및 향후 연구

본 논문은 미래 자리를 미리 확정하지 않고 매시간 대기 작업 전부의 리전과 시작 시각을 다시 배정하는 탄소 인지 스케줄러 CAST를 제안했다. CAST는 마감과 용량 상한을 지키며 상한 없는 실행 절감의 98%를 얻었다(합성). 다른 작업의 계획을 보지 않고 시각을 정하는 설계는 시작 때 확인하지 않으면 작업의 9.0~17.0%가 상한을 넘겼고, CAST보다 많이 줄이는 선행 정책 둘은 상한 없이 돌리면 가장 붐빈 리전에 기준 피크의 2.2배와 9.9배를 요구했다. 상한이 걸리는 자리는 공간 이동이 모이는 캘리포니아와 프랑스로, 상한 없는 실행의 초과 시간 가운데 93.3%가 두 리전에서 났다.

해석과 한계는 다음과 같다. CAST의 이점은 작고 워크로드와 예약 설정에 따라 바뀐다. 합성에서 가장 가까운 설계보다 1.19%p 덜 배출했고 무릎점으로는 같은 배출에 대기 시간이 짧았지만, Azure에서 예약 ρ=13을 두면 다시 풀지 않는 설계가 앞섰다. 회계는 평균 배출계수 위에 있어 한계 배출로 세면 절감이 절반 가까이 줄고, 프랑스 한쪽 대리에서는 다시 풀기의 이점도 사라진다. 설비 효율(PUE), 유휴 전력, 추가 설비의 내재 배출은 넣지 않았다. 리전마다 용량이 정해져 있다고 가정했으며 설비 총량으로는 CASPER가 CAST보다 작다. 실제 클러스터가 아닌 시뮬레이션이고 이론 보장이 없다. 지연 등급은 합성값이고, Azure의 출발 리전과 VM 크기는 모형이며, 실행 시간을 제출 때 안다고 두었다. 위반 0은 이용률 43.2%까지만 확인했고, 인도와 일본 자료는 산정 방식이 바뀐 것으로 보이며, 선행 정책은 원 논문의 자료가 아니어서 원 수치의 재현은 확인하지 못했다.

향후 과제는 실제 예보와 한계 배출계수를 쓰는 회계, 실제 클러스터에서의 검증, Zanotto 등이 남긴 리전 상한의 추정[4], 그리고 슬롯마다 다른 가중치 교환비를 하나로 맞추는 방법이다.

참고문헌

[1] International Energy Agency, Energy and AI, World Energy Outlook Special Report, IEA, Paris, 2025.

[2] T. Sukprasert, A. Souza, N. Bashir, D. Irwin, and P. Shenoy, “On the Limitations of Carbon-Aware Temporal and Spatial Workload Shifting in the Cloud,” Proc. of the 19th European Conf. on Computer Systems (EuroSys), pp. 924-941, Apr. 2024.

[3] T. Bahreini, A. N. Tantawi, and O. Tardieu, “Caspian: A Carbon-aware Workload Scheduler in Multi-Cluster Kubernetes Environments,” Proc. of the 32nd IEEE Int. Symp. on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS), pp. 1-8, Oct. 2024.

[4] M. Zanotto, L. Vicentini, R. Vreto, F. Lumpp, D. Braga, and S. Fiore, “A User-centric Kubernetes-based Architecture for Green Cloud Computing,” arXiv:2509.13325, 2025.

[5] Green Software Foundation, Carbon Aware SDK, v1.8.0 [Online]. Available: https://github.com/Green-Software-Foundation/carbon-aware-sdk (downloaded 2026, Oct. 18)

[6] P. Wiesner, I. Behnke, D. Scheinert, K. Gontarska, and L. Thamsen, “Let's Wait Awhile: How Temporal Workload Shifting Can Reduce Carbon Emissions in the Cloud,” Proc. of the 22nd Int. Middleware Conf. (Middleware), pp. 260-272, Dec. 2021.

[7] W. A. Hanafy, Q. Liang, N. Bashir, D. Irwin, and P. Shenoy, “CarbonScaler: Leveraging Cloud Workload Elasticity for Optimizing Carbon-Efficiency,” Proc. of the ACM on Measurement and Analysis of Computing Systems (POMACS), Vol. 7, No. 3, Article 57, Dec. 2023.

[8] W. A. Hanafy, Q. Liang, N. Bashir, A. Souza, D. Irwin, and P. Shenoy, “Going Green for Less Green: Optimizing the Cost of Reducing Cloud Carbon Emissions,” Proc. of the 29th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Vol. 3, pp. 479-496, Apr. 2024.

[9] K. Xu, D. Sun, H. Tian, J. Zhang, and K. Chen, “GREEN: Carbon-efficient Resource Scheduling for Machine Learning Clusters,” Proc. of the 22nd USENIX Symp. on Networked Systems Design and Implementation (NSDI), pp. 999-1014, Apr. 2025.

[10] A. Souza, S. Jasoria, B. Chakrabarty, D. Irwin, P. Shenoy, A. Bridgwater, A. Lundberg, F. Skogh, and A. Ali-Eldin, “CASPER: Carbon-Aware Scheduling and Provisioning for Distributed Web Services,” Proc. of the 14th Int. Green and Sustainable Computing Conf. (IGSC), pp. 67-73, Oct. 2023.

[11] A. Lechowicz, N. Christianson, B. Sun, N. Bashir, M. Hajiesmaili, A. Wierman, and P. Shenoy, “Learning-Augmented Competitive Algorithms for Spatiotemporal Online Allocation with Deadline Constraints,” Proc. of the ACM on Measurement and Analysis of Computing Systems (POMACS), Vol. 9, No. 1, Article 8, pp. 1-49, Mar. 2025.

[12] S. Hall, F. Micheli, G. Belgioioso, A. Radovanović, and F. Dörfler, “Carbon-Aware Computing for Data Centers with Probabilistic Performance Guarantees,” arXiv:2410.21510, 2024.

[13] A. Radovanović, R. Koningstein, I. Schneider, B. Chen, A. Duarte, B. Roy, D. Xiao, M. Haridasan, P. Hung, N. Care, S. Talukdar, E. Mullen, K. Smith, M. Cottman, and W. Cirne, “Carbon-Aware Computing for Datacenters,” IEEE Transactions on Power Systems, Vol. 38, No. 2, pp. 1270-1280, Mar. 2023.

[14] W. A. Hanafy, L. Wu, D. Irwin, and P. Shenoy, “CarbonFlex: Enabling Carbon-aware Provisioning and Scheduling for Cloud Clusters,” arXiv:2505.18357, 2025.

[15] W. Feng, A. Dasgupta, Z. D. Tekler, S. Ahuja, J. Zheng, and X. Jiang, “COAST: Congestion-Aware Start-Time Recommendations for Carbon-Aware HPC Jobs,” arXiv:2609.05443, 2026.

[16] D. Maji, R. K. Sitaraman, and P. Shenoy, “DACF: Day-Ahead Carbon Intensity Forecasting of Power Grids using Machine Learning,” Proc. of the 13th ACM Int. Conf. on Future Energy Systems (e-Energy), pp. 188-192, Jun.-Jul. 2022.

[17] J. Edmonds, “Submodular Functions, Matroids, and Certain Polyhedra,” in Combinatorial Optimization — Eureka, You Shrink!, LNCS 2570, pp. 11-26, Springer, 2003.

[18] A. Schrijver, Theory of Linear and Integer Programming, Wiley, Chichester, 1986.

[19] A. J. Hoffman and J. B. Kruskal, “Integral Boundary Points of Convex Polyhedra,” in H. W. Kuhn and A. W. Tucker (Eds.), Linear Inequalities and Related Systems, Annals of Mathematics Studies 38, Princeton University Press, pp. 223-246, 1956.

[20] M. Zeleny, “Compromise Programming,” in J. L. Cochrane and M. Zeleny (Eds.), Multiple Criteria Decision Making, University of South Carolina Press, Columbia, pp. 262-301, 1973.

[21] J. Branke, K. Deb, H. Dierolf, and M. Osswald, “Finding Knees in Multi-objective Optimization,” Proc. of the 8th Int. Conf. on Parallel Problem Solving from Nature (PPSN VIII), LNCS 3242, pp. 722-731, Sep. 2004.

[22] Electricity Maps. API Documentation [Online]. Available: https://docs.electricitymaps.com (downloaded 2026, Oct. 12)

[23] Electricity Maps. Methodology [Online]. Available: https://www.electricitymaps.com/data/methodology (downloaded 2026, Sep. 22)

[24] Microsoft. Azure Network Round-Trip Latency Statistics [Online]. Available: https://learn.microsoft.com/en-us/azure/networking/azure-network-latency (downloaded 2026, Sep. 22)

[25] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, "Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms," Proc. of the 26th ACM Symp. on Operating Systems Principles (SOSP), pp. 153-167, Oct. 2017.

[26] Microsoft Azure, Azure Public Dataset V2, 2019 [Online]. Available: https://github.com/Azure/AzurePublicDataset (downloaded 2026, Oct. 17)